RAKSHANA 24/7

□□□□□ 24/7 • Light That Travels Ahead of the Harm

PRODUCT REQUIREMENTS DOCUMENT

Phase 2: Technical Architecture, System Design & Security

Hackathon	Alliance One 2026 – SHASTRA CodeSangram
Problem ID	AO/SHASTRA/CS/26/001
Category	Digital Safety & Cyber-Hygiene for Women
Version	v2.0 | Phase 2 Blueprint
Status	Active — Phase 2 Technical Evaluation

1. Product Overview

RAKSHANA 24/7 is a proactive digital threat intelligence platform built exclusively for women's safety. Unlike every existing tool in this space — which require a woman to actively report an incident after it has already happened — RAKSHANA 24/7 monitors public digital surfaces continuously and fires an alert before harassment escalates.

The name comes from the Telugu word for "light." The metaphor is intentional: light travels ahead of whatever is coming. That is exactly what this platform does — it sees the signal before the woman feels the harm.

1.1 The Core Thesis

Most women do not know they are being targeted until the harassment reaches their inbox, their phone, or their front door. By that point, evidence has spread, damage is done, and reporting becomes reactive and painful. RAKSHANA 24/7 inverts this cycle entirely.

What we build: A background threat-monitoring service that watches publicly available internet surfaces for the user's digital fingerprint — phone number, photos, social handles — and raises a graded alert the moment that fingerprint appears in a suspicious context.

What makes this different: Every alert is paired with an actionable response path — what to screenshot, which legal sections apply, how to file an anonymous complaint, who to notify. The alert and the action are one atomic unit.

1.2 Problem Statement Summary

Gap	What Currently Exists	What RAKSHANA 24/7 Solves
Detection Gap	No early-warning system. Threats spread for days undetected.	Continuous public surface scan — Telegram, paste sites, reverse image search.
Accessibility Gap	Portals are English-only, desktop-first, FIR-number required upfront.	Mobile-first, vernacular UI (Telugu/Kannada/Hindi), zero bureaucracy.
Response Gap	Post-report blackhole. No tracking, no counselling, no next step.	Every alert = legal context + evidence guide + one-tap anonymous cyber report.

2. System Architecture

RAKSHANA 24/7 is designed as a three-tier architecture: a lightweight mobile-first frontend, a Flask-based REST API backend, and a background worker system that handles continuous monitoring independently of user interactions. This separation is intentional — the monitoring must never be blocked by frontend activity, and the user-facing app must never be slow because of a background scan.

2.1 Architecture Overview

The system has four distinct layers, each with a clearly scoped responsibility:

- Presentation Layer — HTML/CSS/JS PWA + mobile-responsive web app
- API Layer — Flask REST API, handles auth, user management, alert delivery
- Worker Layer — Celery task queue + Redis broker for async background scanning
- Data Layer — SQLite (dev) / PostgreSQL (prod), SQLAlchemy ORM, encrypted fields

2.2 Why This Architecture — The Reasoning

Flask over Django: RAKSHANA 24/7's backend is not a traditional CRUD app. It is a coordination layer between a frontend and multiple async scanning workers. Flask's lightweight, explicit routing makes it far easier to build clean REST endpoints without carrying the weight of Django's ORM assumptions and admin overhead. We control exactly what runs and when.

Celery + Redis over cron jobs: Monitoring thousands of users' digital fingerprints cannot happen in a single synchronous loop. Celery allows us to schedule and distribute individual scan tasks per user, retry on failure, and fan out workload across multiple workers as we scale. Redis as the message broker is fast, memory-resident, and perfectly suited for this use case. A simple cron job would block, fail silently, and not scale beyond 50 users.

SQLite for hackathon, PostgreSQL for production: SQLite requires zero infrastructure setup, making it perfect for rapid prototype and demo. When RAKSHANA 24/7 graduates to production, swapping to PostgreSQL is a one-line config change in SQLAlchemy — no code changes. This is a deliberate, forward-thinking decision.

PWA over native app: Building separate iOS and Android apps is expensive, slow to update, and requires app store approval cycles. A Progressive Web App delivers app-like behaviour — offline cache, push notifications, home screen install — through the browser. Critically, a woman in rural Andhra Pradesh with a basic Android phone on 3G can install RAKSHANA 24/7 by visiting a URL. No Play Store, no download friction.

2.3 Component Interaction Diagram (Textual)

PWA Frontend	→	Flask API	→	Celery Workers	→	External APIs
User registration Dashboard Alert inbox Report filing		Auth endpoints Fingerprint store Alert management WebSocket push		Telegram scan Image search Paste monitor Pattern scoring		Telethon Google Vision Twilio/WA Cyber cell API

3. Technology Stack — With Justification

Every technology in this stack was chosen because it fits RAKSHANA 24/7 specifically, not because it is popular. The reasoning for each is critical context for the Phase 2 evaluation.

3.1 Backend

Technology	Version	Why RAKSHANA 24/7 specifically uses it
Python 3.11	3.11+	Telethon (Telegram API), BeautifulSoup4, NLTK are all Python-native. One language across all layers = zero context switching.
Flask	3.x	Minimal surface area. We need API routes + auth + WebSocket push. Flask does exactly this with no excess baggage. Blueprint-based modular design matches our component separation.
Telethon	1.36	The ONLY Python library that lets you read public Telegram channels programmatically without a bot token — just a phone + API key. This is our primary threat signal source.
BeautifulSoup4	4.12	Lightweight HTML parser for scraping paste sites (Pastebin, Ghostbin) for phone number and name mentions. Regex alone misses context — BS4 lets us parse document structure.
NLTK / spaCy	Latest	Named Entity Recognition to distinguish a match that is "John called Priya" vs "Priya's number leaked." Context-aware scoring, not just keyword matching.
Celery	5.x	Distributed task queue. Each user gets their own periodic scan task. Failed tasks retry automatically. Workers scale horizontally. This is the engine of proactive monitoring.
Redis	7.x	In-memory message broker for Celery. Also used for caching recent scan results (avoid redundant API calls) and session management.

3.2 Database

Technology	Stage	Justification
SQLite + SQLAlchemy	Hackathon/Dev	Zero setup. SQLAlchemy ORM means our models are database-agnostic. Swapping to Postgres is a single connection string change. No schema migration needed.
PostgreSQL	Production	Full ACID compliance, row-level security, native JSON columns for threat metadata, and PostGIS for future location-based features. The logical graduation from SQLite.

Encrypted Fields	Both	Phone numbers, photos, and profile data are encrypted at rest using SQLAlchemy-Utils EncryptedType with AES-256. A DB breach exposes ciphertext only.

3.3 Frontend

Technology	Detail	Justification
HTML5 + CSS3 + Vanilla JS	ES6+	No React, no Vue. RAKSHANA 24/7's UI is structurally simple — dashboard, alert inbox, registration. The complexity is in the backend. A framework adds 200KB+ bundle for no UX gain.
Progressive Web App	Service Worker + Manifest	Offline support, push notifications, home screen install. Works on basic Android on 3G. The target user does NOT have the luxury of high-end devices.
i18n (Vernacular)	Telugu, Kannada, Hindi	Not just translation — culturally contextualised language. "Cyberstalking" means nothing; "someone is tracking your location without permission" does. JSON locale files, auto-detect browser language.
Threat Dashboard	Canvas.js / Chart.js	Visual threat score timeline. A graph showing "your threat level over the last 7 days" is far more actionable than a list of raw alerts. Users understand trends, not log entries.

4. Data Flow & Security Architecture

Data security is not an afterthought in RAKSHANA 24/7 — it is the product. A woman is trusting us with her phone number and her photo. If that data is compromised, we have made her more vulnerable, not less. Every design decision in this section is made with that responsibility in mind.

4.1 Data Flow — Registration to Alert

The following describes the complete data journey from registration through to threat alert delivery:

1. User opens RAKSHANA 24/7 PWA. Completes registration: phone number (optional), one selfie photo, social media handles. All transmitted over HTTPS/TLS 1.3.
2. Flask API receives the registration payload. Phone number is hashed (SHA-256) for scanning use. Photo is processed through Google Vision API to extract facial feature vector — the raw photo is NOT stored on our servers post-processing. Only the feature vector is retained, encrypted with AES-256.
3. SQLAlchemy writes the encrypted user profile to SQLite/PostgreSQL. A Celery periodic task is registered for this user with a configurable scan interval (default: every 4 hours).
4. Celery worker picks up the scan task. Three sub-tasks run in parallel: (a) Telegram public channel scan via Telethon, (b) Paste site scrape via BS4, (c) Reverse image search via Google Vision.
5. Each sub-task returns raw matches. The NLP scoring module (spaCy NER) evaluates context: is the phone number appearing in a casual conversation or being broadcast with threatening language? Threat score is computed on a 0–100 scale.
6. If score > 40 (watch threshold): User receives a passive dashboard update. If score > 70 (alert threshold): Push notification fires via Twilio/WhatsApp with vernacular alert text + action guide. If score > 90 (critical): Trusted contact is notified (if configured).
7. Every alert includes: screenshot instructions, relevant ITA 2000 sections, pre-filled cybercrime.gov.in form link, and optional one-tap anonymous submission.

4.2 Security Architecture

Layer	Threat	Our Mitigation
Transport	MITM, data interception	TLS 1.3 enforced, HSTS headers, no HTTP fallback
Data at Rest	DB breach exposes PII	AES-256 field-level encryption on phone, photo vector, handles
Authentication	Account takeover	JWT with short expiry + refresh token rotation, optional OTP
Anonymous Reporting	Report traced back to user	Reports submitted via anonymising proxy. No IP logged server-side

Photo Data	User photo stored + leaked	Raw photo deleted post feature-extraction. Only encrypted vector retained
Crawler Visibility	Our scan IPs get banned	Rotating residential proxies + exponential backoff + respectful rate limits
False Positives	Panic from wrong alerts	Graduated scoring (0-100), never binary. Explanation always shown alongside score

4.3 Privacy Principles

- Minimum Data Collection: We ask only what is needed for monitoring. We do not require name, address, or government ID.
- User Owns Their Data: Full export and delete available at any time. Deletion cascades to all encrypted records within 24 hours.
- No Third-Party Selling: User profile data is never shared with or sold to any third party, advertiser, or government body without explicit legal process.
- Scan Transparency: Users can see exactly what was scanned, when, and what was found. No black boxes.

5. UI/UX Design Philosophy & Approach

The RAKSHANA 24/7 interface has to work for two very different users simultaneously: an urban, tech-savvy college student in Hyderabad, and a first-generation smartphone user in a rural village in Andhra Pradesh. Both need to feel safe, in control, and not overwhelmed.

Our design principle is: "Don't make her think. Make her feel protected." Every screen should communicate safety, not complexity.

5.1 Design System

Element	Choice	Reasoning
Primary Color	#7B2FBE (Deep Purple)	Purple is universally associated with protection, royalty, and strength in South Asian cultures. Not red (fear), not blue (corporate).
Accent	#FF6B9D (Pink)	Signals femininity without being patronising. Used sparingly for CTAs and alert highlights.
Alert Colors	Green / Amber / Red scale	Universal traffic-light language. Requires no explanation across languages or literacy levels.
Typography	Noto Sans (Latin + Telugu/Kannada/Hindi)	Google's Noto family has complete Unicode coverage for all South Asian scripts. Single font, all languages.
Touch Targets	Minimum 48x48px	WCAG AA standard. Critical for users with tremors or lower motor precision on budget smartphones.
Information Architecture	3-tab bottom nav	Home (status), Alerts, Settings. Never more than 2 taps to any critical action.

5.2 Key Screens & User Flows

Screen 1 — Onboarding / Registration

The registration screen is designed to feel like a conversation, not a form. Step-by-step progressive disclosure: first, what is your phone number? Then, optional photo. Then, social handles. Each step has a plain-language explanation of what it is used for and why. "Your photo helps us find if someone is using it without your permission."

Screen 2 — Protection Status Dashboard

The home screen has one dominant element: a large circular shield indicator showing the current threat score (0-100). Green (safe), amber (watching), red (alert). Below it: the last scan timestamp, and recent activity feed. No jargon. "Your digital presence was last scanned 2 hours ago. No threats found."

Screen 3 — Alert Detail Screen

When an alert fires, the screen shows: what was found, where it was found (Telegram group name, paste site URL), a translated explanation of why this is a threat, and three action buttons: "Screenshot & Save Evidence," "File Anonymous Report," "Notify Trusted Contact." The legal section reference appears in a small card at the bottom — there for those who need it, invisible to those who don't.

Screen 4 — Threat Timeline Dashboard

A 7-day Chart.js line graph showing threat score trend. This is the analytical differentiator. A woman can see if her score spiked on a specific date — maybe after she posted in a public group, maybe after she rejected someone. Pattern awareness is power.

5.3 Figma Design Approach

The UI is being prototyped in Figma using the following component library structure:

- Atomic Design System: Tokens (colors, spacing, typography) → Components (buttons, cards, alerts) → Pages
- Mobile-first frames: 390×844 (iPhone 14 base) and 360×800 (mid-range Android base)
- Auto-layout everywhere: Components stretch/compress for different screen sizes without manual adjustment
- Accessibility annotations: Contrast ratios, touch target sizes, screen reader labels all documented in Figma
- Vernacular variants: Each screen has a Telugu language variant to validate layout with longer text strings

Design tool stack: Figma (primary prototyping) + Figma AI (component suggestions) + Cairo / Google Fonts for typography testing. Handoff to developers via Figma Dev Mode — CSS values, spacing tokens, and asset exports are automatically available without a separate spec document.

6. Scalability & Application Complexity

The judges specifically evaluate scalability and the complexity of what we are building. Here is why RAKSHANA 24/7 is technically non-trivial, and how it scales.

6.1 What Makes This Complex

- Real-time crawling without getting banned: Telegram rate limits, IP rotation, backoff strategies — this is not a tutorial CRUD app.
- NLP threat scoring in context: The difference between "Priya's number is +91-XXXX" in a contact-sharing group vs the same number in a doxxing thread requires NER + context window analysis.
- Encrypted PII at rest with search capability: You cannot run a LIKE query on encrypted data. We use deterministic encryption for search fields and probabilistic for storage. This is an advanced pattern.
- Multi-source signal fusion: Telegram + paste sites + reverse image search all return different data shapes. Normalising these into a unified threat score requires a designed pipeline, not ad-hoc parsing.
- Graduated alert system: Binary "safe/unsafe" is useless. A 0-100 graduated score with threshold-based escalation requires thoughtful calibration to avoid false positives that destroy user trust.

6.2 Scalability Strategy

Scale Dimension	Current (Hackathon)	Production Path
Users	100 demo users, SQLite, single worker	PostgreSQL, Celery worker pool, horizontal scaling on PythonAnywhere
Scan Volume	4-hour intervals, sequential	Parallel workers, priority queue (high-risk users scan more frequently)
Alert Delivery	Twilio SMS/WhatsApp	Add Firebase FCM for push notifications at zero marginal cost
Data Volume	SQLite file	PostgreSQL with table partitioning by user_id for scan_results table
Languages	Telugu + English	JSON locale files — adding a new language is adding one JSON file, no code change

7. Hackathon MVP Scope

What we commit to demonstrating in Phase 2 evaluation:

8. User registration with phone number and photo upload
9. Digital fingerprint extraction and encrypted storage in SQLite
10. Live Telegram public channel scan using Telethon — demo will show a real scan result
11. Threat scoring engine returning a 0-100 score with contextual explanation
12. Alert delivery via WhatsApp/SMS using Twilio
13. PWA frontend with protection status dashboard + alert inbox
14. Vernacular toggle (English ↔ Telugu) on live demo
15. Threat timeline chart showing score trend

What is out of scope for the hackathon but designed for in the architecture: reverse image search pipeline, trusted contact network, full anonymous reporting proxy, production PostgreSQL migration.

8. Why RAKSHANA 24/7 Wins

Other teams will build a reporting chatbot. We are building a sentinel. The architecture choices here — Celery for async monitoring, Telethon for Telegram access, NLP for context-aware scoring, PWA for accessibility, graduated alerts for trust — are not just technically sound. They are the only way this problem can actually be solved.

The woman who needs RAKSHANA 24/7 is not waiting for a better reporting form. She needs something that is already watching when she is asleep.

"Light that travels ahead of the harm. □□□□□ 24/7"